Unidad N°8

Recursividad, Búsqueda, Ordenamiento

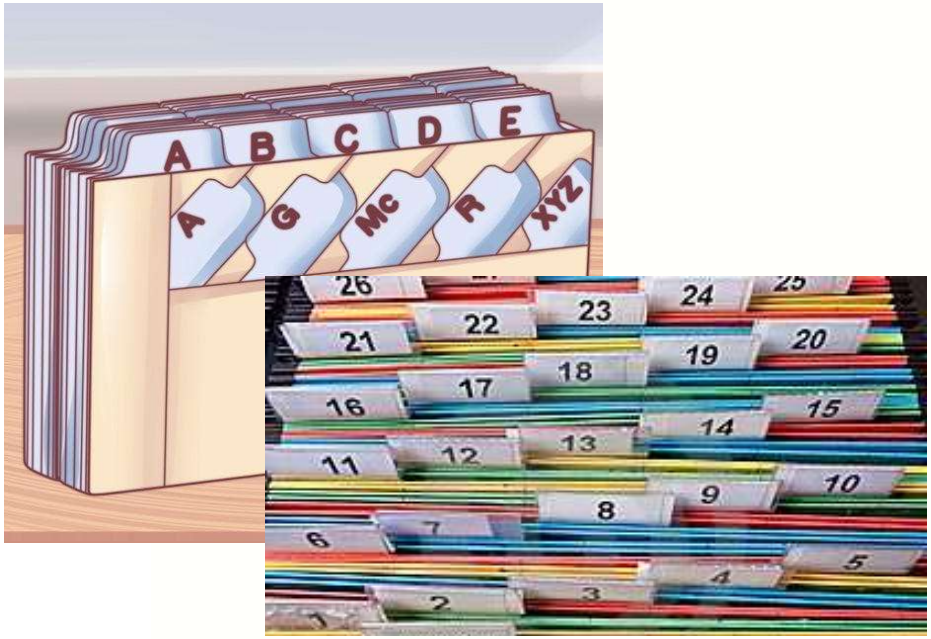
UTN VILLA MARÍA

ALGORITMOS FUNDAMENTALES

- De Ordenamiento
- De Búsqueda
- De Recorrido

ALGORITMOS FUNDAMENTALES

De Ordenamiento

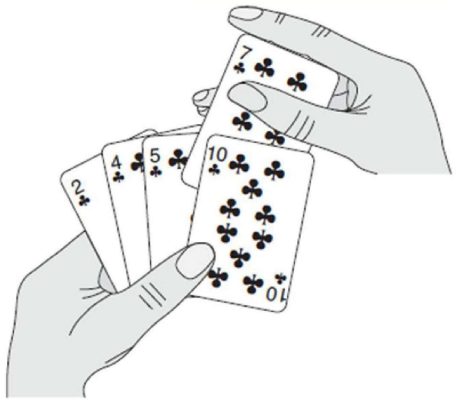


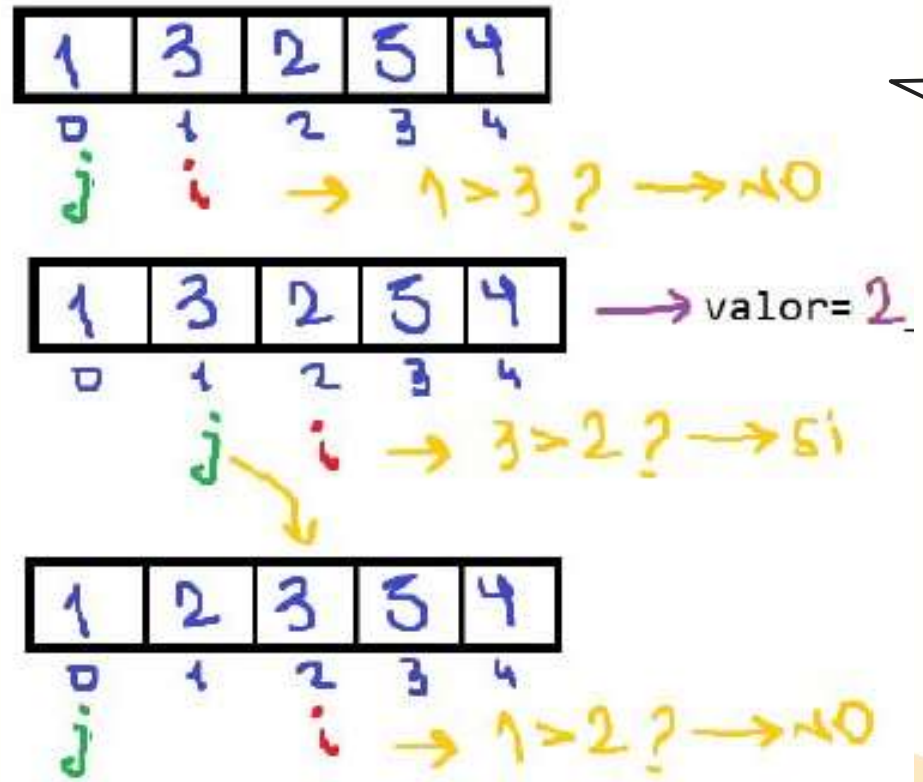
- ✔ Sirven para dar un orden determinado a los elementos de una lista.
- ✔ Ordenar listas en orden alfabético u orden numérico. -> Muy usual.
- ✔ El orden puede ser ascendente (de menor a mayor) o descendente (de mayor a menor).
- ✔ Algunos algoritmos de ordenación son:
  - **Ordenamiento por Inserción.**
  - **Algoritmo de la Burbuja.**
  - **Ordenamiento por Selección.**
  - **Algoritmo Quick-Sort**

https://www.youtube.com/watch?v=5kVQ8kf52K4

De Ordenamiento

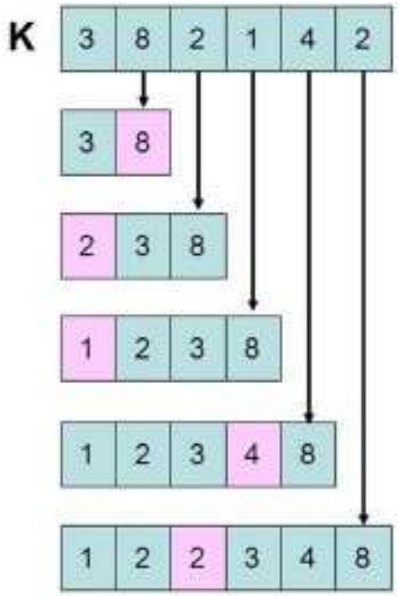
Ordenamiento por inserción





1º ITERACIÓN

2º ITERACIÓN



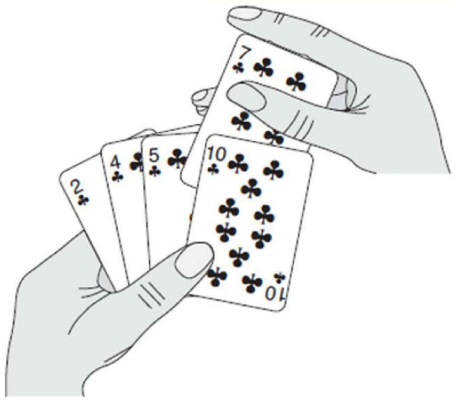
El pseudocódigo para este algoritmo es el siguiente:

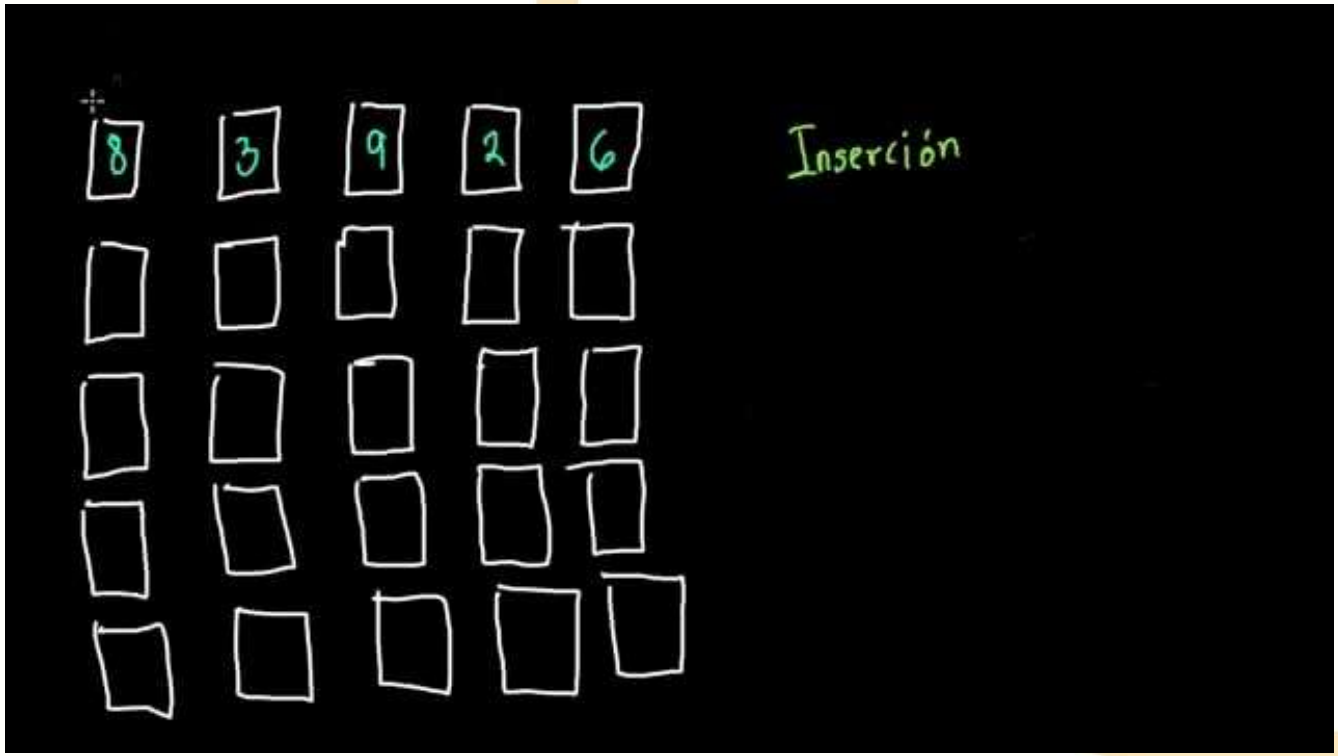
```
BEGIN insercion (A: lista de elementos)
  FOR (ENTERO i = 1; i < longitud(A); i++) :
    ENTERO valor = A[i]
    ENTERO j = i-1

    WHILE (j >= 0 && A[j] > valor)
    DO:
      A[j+1] = A[j]
      j--
    END_WHILE

    A[j+1] = valor
  END_FOR
END
```

https://www.youtube.com/watch?v=5kVQ8kf52K4

De Ordenamiento

Ordenamiento por inserción



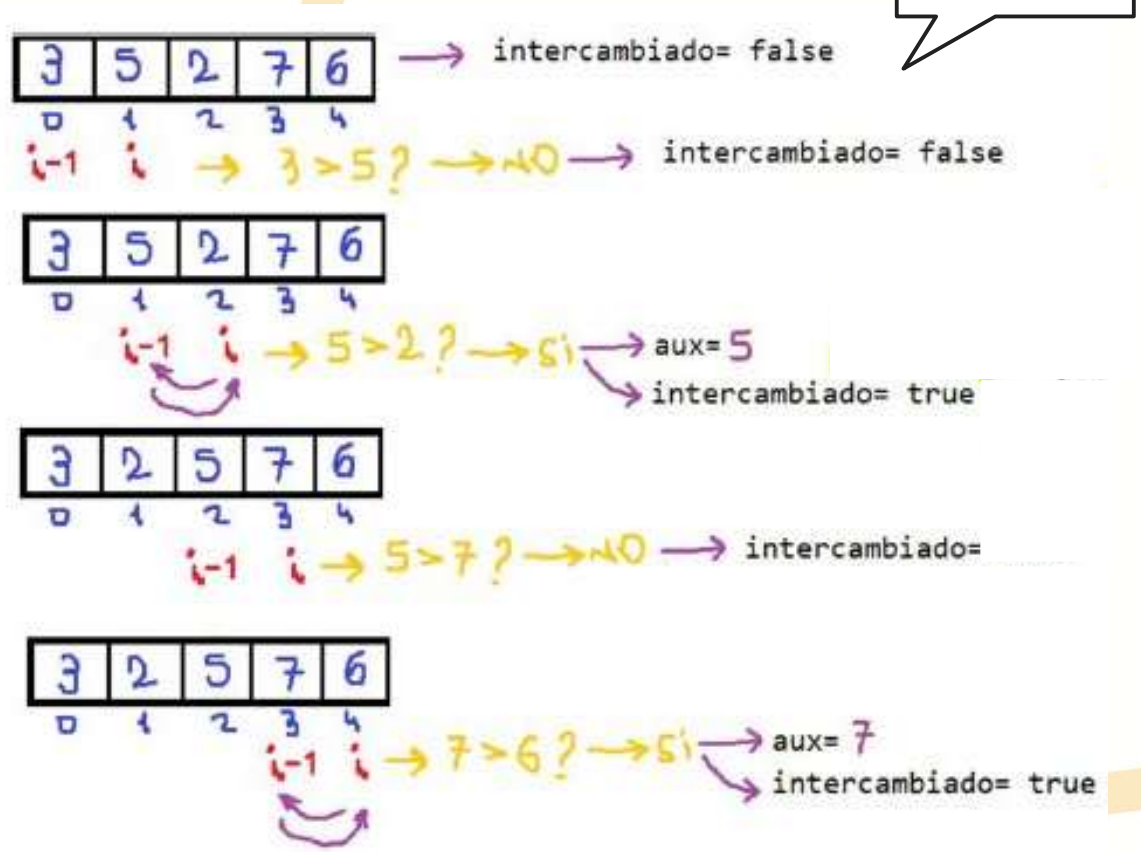


https://www.youtube.com/watch?v=L3d48etbseY

De Ordenamiento

Algoritmo de la burbuja

- ✔ Este algoritmo consiste en comparar cada elemento de la lista con el siguiente (por parejas), si no están en el orden correcto, se intercambian entre sí sus valores.
- ✔ El valor más pequeño flota hasta el principio de la lista.



2 3 4 5 1	No ordenado
2 3 4 5 1	2 < 3, ok
2 3 4 5 1	3 < 4, ok
2 3 4 5 1	4 < 5, ok
2 3 4 5 1	5 > 1, intercambiar
2 3 4 1 5	2 < 3, ok
2 3 4 1 5	3 < 4, ok
2 3 4 1 5	4 > 1, intercambiar
2 3 1 4 5	2 < 3, ok
2 3 1 4 5	3 > 1, intercambiar
2 1 3 4 5	2 > 1, intercambiar
1 2 3 4 5	Ordenado

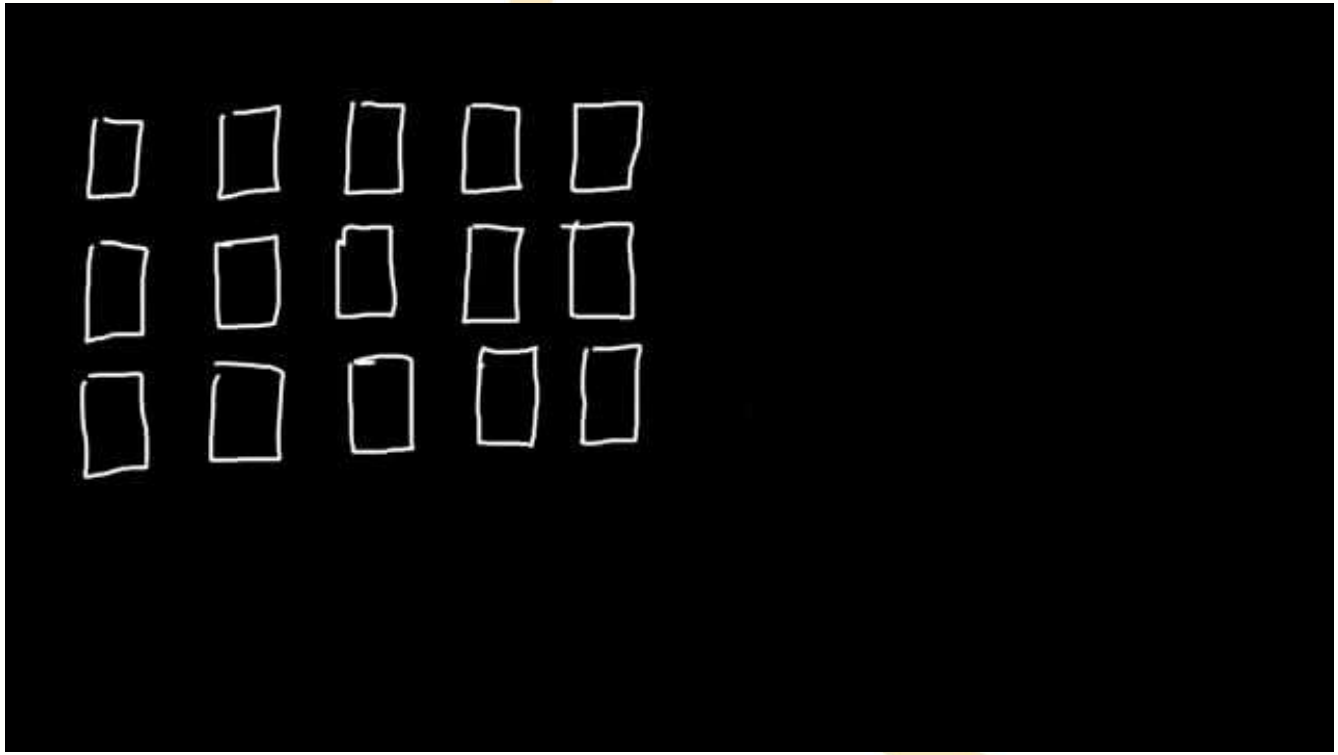
El pseudocódigo para este algoritmo es el siguiente:

```
BEGIN burbuja (A: lista de elementos)
  n = longitud(A)
  DO:
    intercambiado = falso

    FOR (ENTERO i = 1; i < n; i++)
      // si este par no está ordenado
      IF (A[i-1] > A[i]) THEN:
        // los intercambiamos y recordamos que algo ha cambiado
        ENTERO aux = A[i-1]
        A[i-1] = A[i]
        A[i] = aux

        intercambiado = verdadero
      END_IF
    END_FOR
  WHILE (intercambiado == verdadero)
END
```

https://www.youtube.com/watch?v=L3d48etbseY

De Ordenamiento

Algoritmo de la burbuja

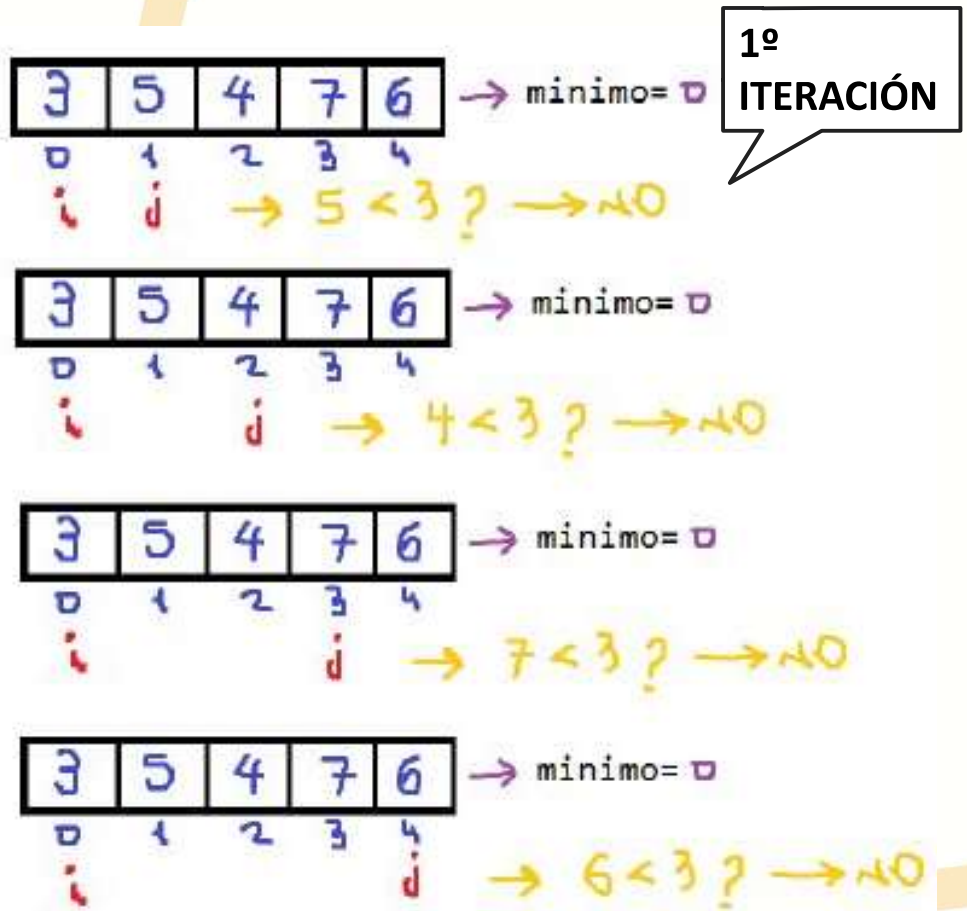


https://www.youtube.com/watch?v=l0YwcUJB3vo

De Ordenamiento

Ordenamiento por Selección

- ✔ Similar al método de la burbuja.
- ✔ Primero recorre toda la lista buscando el **menor/mayor** de todos los elementos.
- ✔ En la siguiente iteración se recorre nuevamente la lista pero comenzando en **el segundo elemento**.

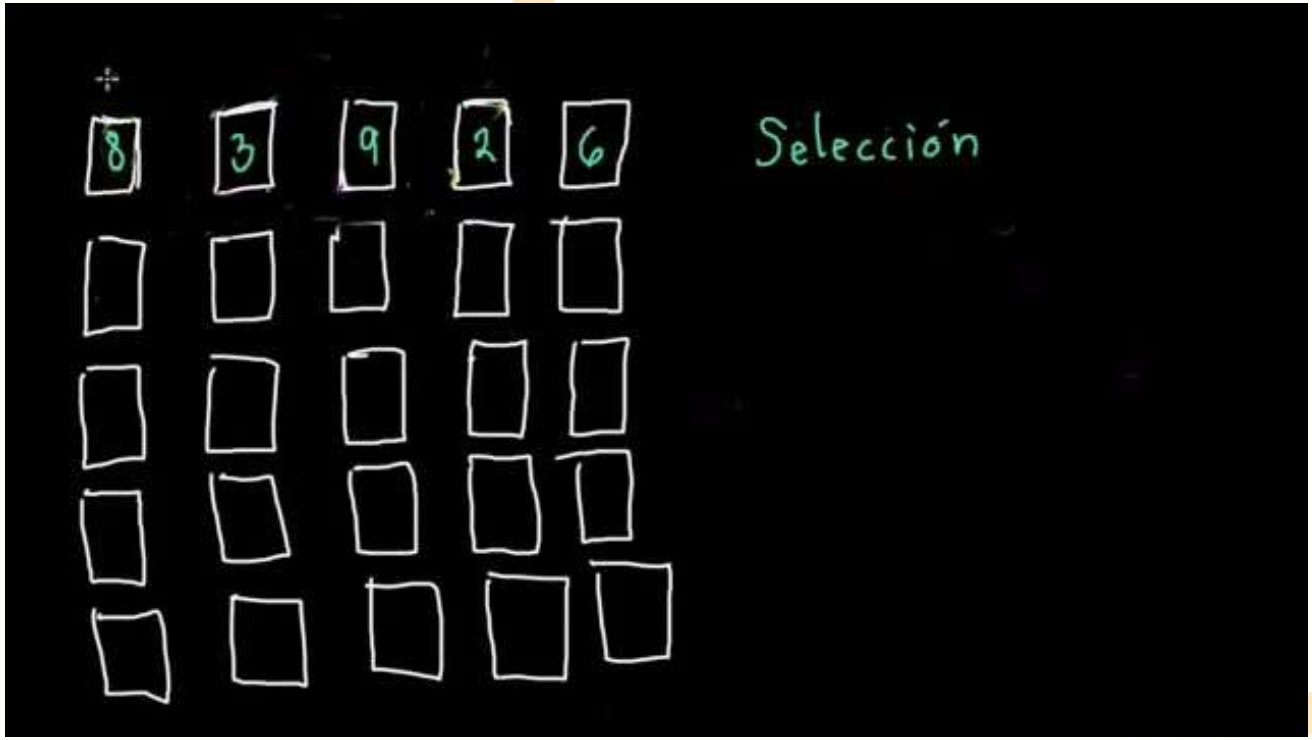


```
BEGIN selección (A : lista de elementos )
 n = longitud(A)
  FOR (ENTERO i = 0; i < n - 1; i++)
  ENTERO mínimo = i
  FOR (ENTERO j = i+1; j < n; j++)
   // si este par no está ordenado
   IF (A[j] < A[mínimo]) ENTONCES:
    // encontramos un nuevo mínimo
    mínimo = j
   END_IF
  END_FOR
  // intercambiamos el actual con el mínimo encontrado
  ENTERO aux = A[minimo]
  A[minimo] = A[i]
  A[i] = aux
 END_FOR
END
```

https://www.youtube.com/watch?v=l0YwcUJB3vo

De Ordenamiento

Ordenamiento por Selección



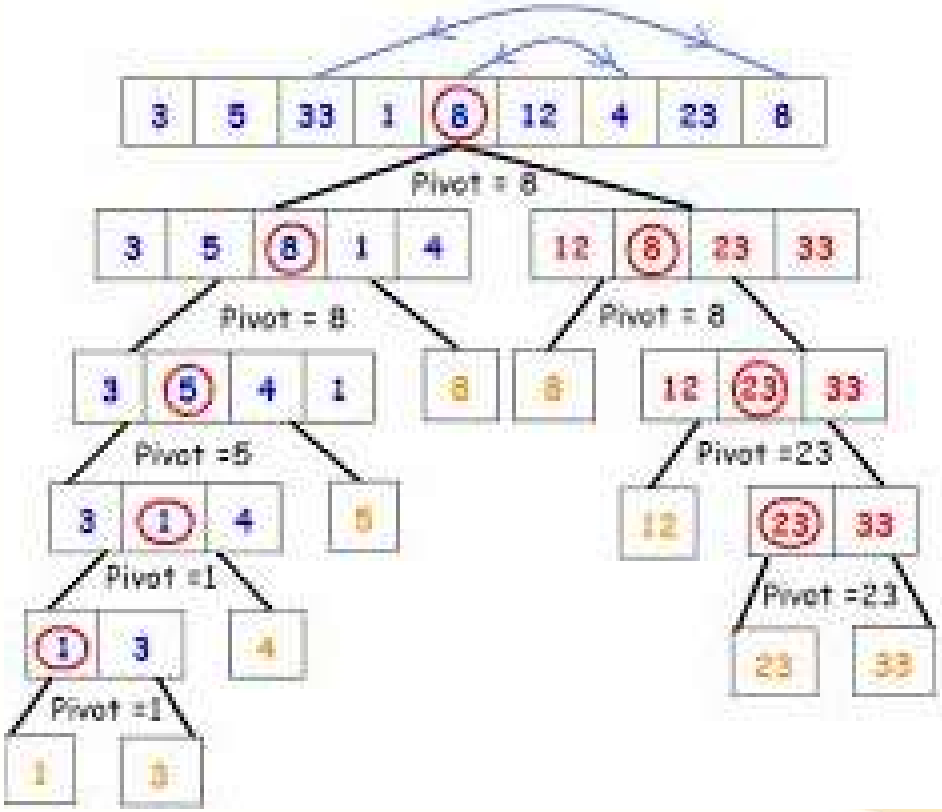
De Ordenamiento

Algoritmo Quick-sort

- ✔ Este tipo de algoritmos se basa en la técnica "divide y vencerás".
- ✔ Pasos del algoritmo:
  - Elegir el elemento **pivote**.
  - Resituar los demás elementos de la lista a cada lado del pivote, de manera que a un lado queden todos los menores que él, y al otro los mayores. En este momento, el pivote ocupa exactamente el lugar que le corresponderá en la lista ordenada.
  - La lista queda separada en dos sublistas, una formada por los elementos a la izquierda del pivote, y otra por los elementos a su derecha.
  - Repetir este proceso para cada sublista mientras éstas contengan más de un elemento.

De Ordenamiento

Algoritmo Quick-sort



De Búsqueda

- ✔ La búsqueda de un elemento dado en una lista.
- ✔ Tarea muy usual.
- ✔ Dos algoritmos típicos que realizan esta tarea son:
  - ✔ Búsqueda **secuencial -> LISTAS NO ORDENADAS**
  - ✔ Búsqueda **binaria** o dicotómica. **-> LISTAS ORDENADAS**

De Búsqueda

Búsqueda secuencial

María	Luis	Juan	Alan	Sofía
0	1	2	3	4

✔ Recorre secuencialmente la lista: Comienza en la primera posición de la lista y se detiene cuando encuentra el **elemento buscado** o bien se alcanza el **final de la lista** sin haberlo encontrado.

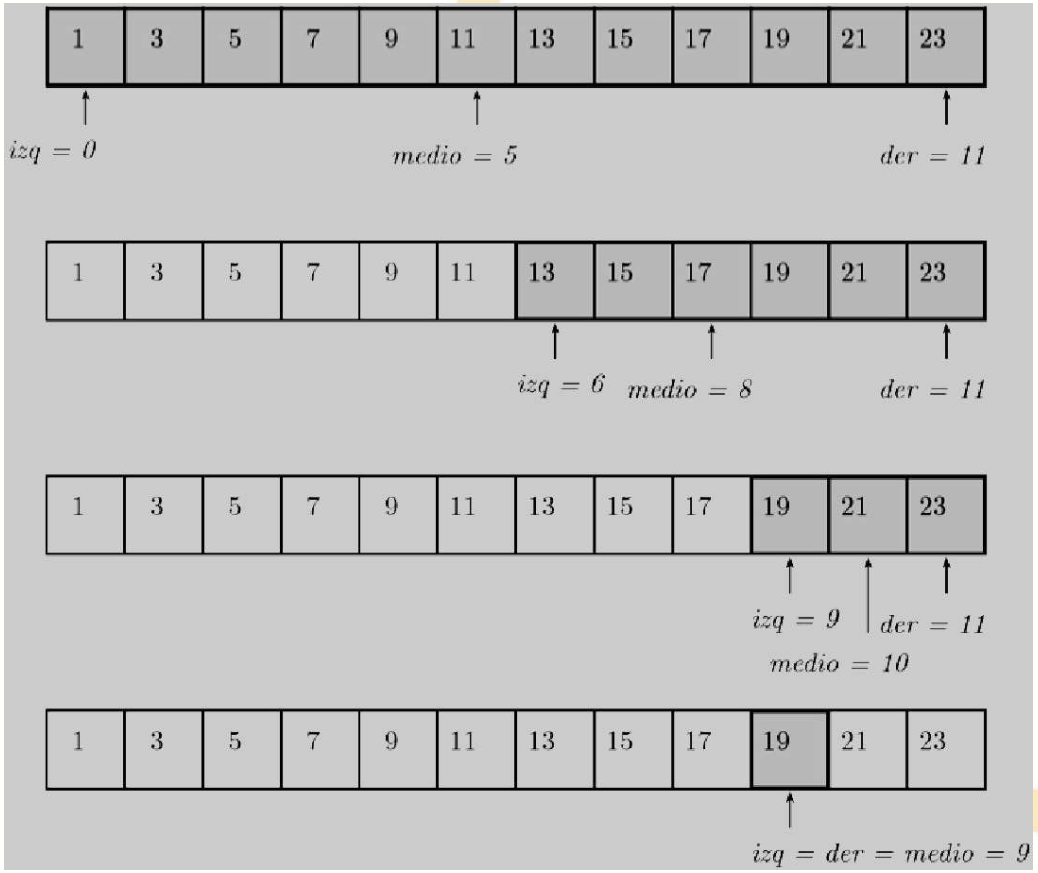
```
BEGIN busquedaSecuencial (L: lista de alumnos, a: alumno buscado )
INTEGER n = longitud(L)
BOOLEAN seEncontró= falso; // si es el alumno a.
    FOR (INTEGER i = 1; i < n - 1; i++)
        IF (L[i] == a)
          THEN: seEncontró = verdadero;
        END_IF
    END_FOR
END
```

De Búsqueda

Búsqueda Binaria

Definir: izq, der, medio

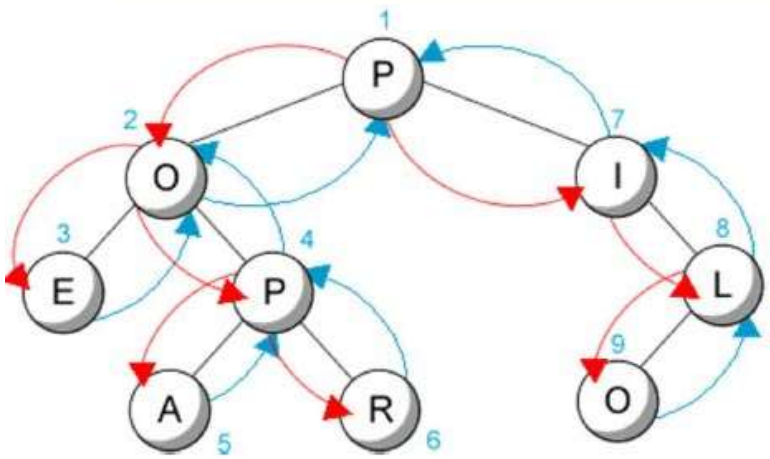
Buscamos el Nº 19



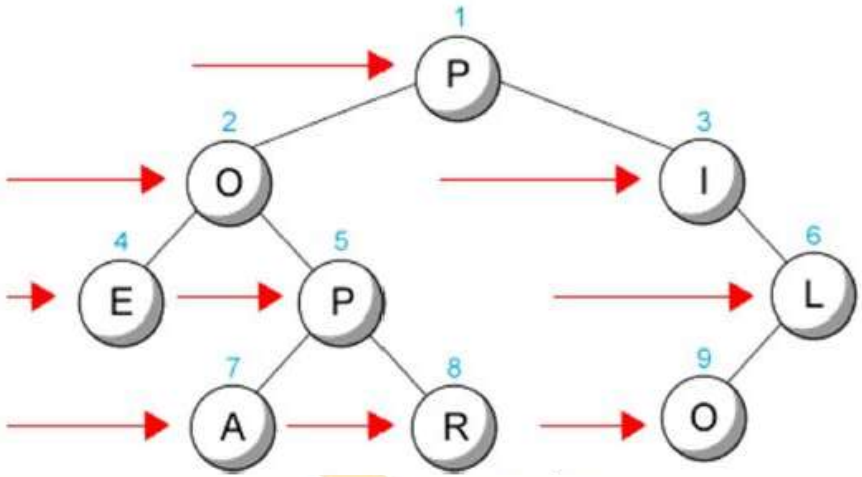
- ✔ Se dispone de una lista ordenada.
- ✔ **Aprovecha que la lista está ordenada para ir disminuyendo el espacio de búsqueda.**

De Recorrido

RECORRIDO EN PROFUNDIDAD



RECORRIDO EN ANCHURA



Para la próxima clase

- **Resolver los enunciados VPL propuestos en la plataforma como obligatorios**